Evolutionary Intelligence (2020) 13:537–549
https://doi.org/10.1007/s12065-020-00355-2

RESEARCH PAPER

Difficult first strategy GP: an inexpensive sampling technique to improve the performance of genetic programming

Muhammad Quamber Ali[1] · **Hammad Majeed**[1]

Received: 21 January 2019 / Revised: 16 December 2019 / Accepted: 25 January 2020 / Published online: 14 February 2020
© Springer-Verlag GmbH Germany, part of Springer Nature 2020

Abstract
Genetic programming (GP) is a top performer in solving classification and clustering problems, in general and symbolic regression problems, in particular. GP has produced impressive results and has outperformed human generated results for 76 different problems taken from 22 different fields. There remain a number of significant open issues despite its impressive results. Among them are high computational cost, premature convergence and high error rate. These issues must be addressed for GP to realise its full potential. In this paper a simple and cost effective technique called Difficult First Strategy-GP (DFS-GP) is proposed to address the aforementioned problems. The proposed technique involves pre-processing and sampling steps. In the pre-processing step, *difficult* to evolve data points by GP from the given data set are marked and in the sampling step, they are introduced in the evolutionary run by using two newly defined sampling techniques, called difficult points first and difficulty proportionate selection. These techniques are biased towards selecting difficult data points during the initial stage of a run and of easy points in the latter stage of a run. This ensures that GP does not ignore difficult-to-evolve data points during a run. Experiments have shown that GP coupled with DFS avoids premature convergence and attained higher fitness than standard GP using same fitness evaluations. Performance of the proposed technique was evaluated on three commonly known metrics, which are convergence speed, fitness and variance in the best results. Our results have shown that the proposed setups had achieved 10–15% better fitness values than Standard GP. Furthermore, the proposed setups had consistently generated better quality solutions on all the problems and utilized 30–50% less computations to match the best performance of Standard GP.

Keywords Difficult first strategy · Genetic programming · Pre-processing · Sampling techniques · Dataset sampling · Machine learning

1 Introduction

Genetic programming (GP) is an evolutionary algorithm that is used to evolve expressions by exploring and picking the best relation between the functions and the terminals. The exploration is a supervised learning process and is guided by the difference between the expected output and the output of the evolved expressions (error). Generally, this error is computed on complete input data set. Some of the data points are easy to evolve by GP, such as points on a straight line as compared to data points on a curved line. In this work, we calculated difficulty of a data point by the curvature of the curve it is on. Figure 1 explains this concept in detail. Blue plot is the expected plot and red is the the plot generated by the best-of-run individual evolved by GP. The data points in the green box are marked difficult because GP has found it difficult (higher error) to evolve while all points outside the box are marked easy. The input data set contains training cases with varying difficulty levels.

Our experimental results have shown that the difficult training cases are mostly ignored by GP (see Sect. 3 for details). This is usually the case when the difficult points are few in number. In these situations, GP improves its solutions by focussing on the majority (easy) points and ignoring the difficult cases. In the latter stages of a run, GP finds it hard to evolve these difficult points after converging to incomplete solutions [15]. This is one of the reasons of convergence

✉ Hammad Majeed
hammad.majeed@nu.edu.pk

Muhammad Quamber Ali
quamber@gmail.com

[1] Department of Computer Science, National University of Computer and Emerging Sciences, Islamabad, Pakistan

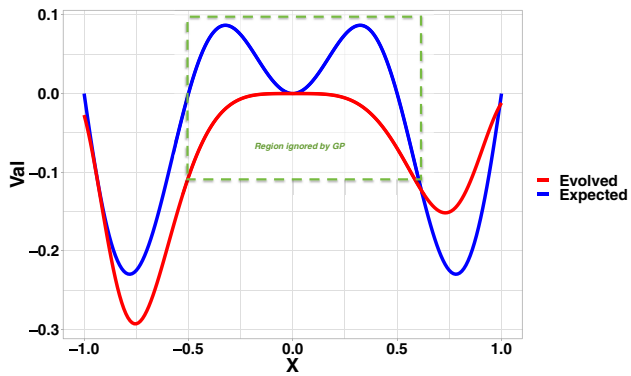


Fig. 1 An example of a bad fit of the best of a run individual

to a lower fitness and waste of computation without fitness improvement in the latter stages of a run. Some researchers have suggested that the performance of GP can be improved by using fraction of input training data set [6, 7, 13, 14]. This paper is based on these findings and extends these work by proposing a new Difficult First Strategy (DFS) to improve the performance of GP.

This paper is organized as follows. Section 2 covers previously proposed techniques on fitness case sampling. Empirical proof of the struggle of Standard GP is discussed in Sect. 3. Our proposed technique is discussed in detail in Sect. 4. Section 5 discusses the experiment setup, and is followed by results and analysis in Sect. 6. Section 7 concludes this work with a possible future direction.

2 Related work

Researchers have used sampling in many ways to achieve different objectives. Mostly the focus of these methods were to avoid over-fitting accompanied with bloat control. But the drawback associated with these methods was fitness reduction. However, we found that work done in co-evolved sampling [11] while reducing fitness cases achieved same/better results. Sampling was done for the sake of generalization and to divert focus on some areas where population was struggling at that moment.

Sampling techniques that are random in nature are Interleaved Sampling, Random Sampling Technique and Data Migration with multiple populations. Interleaved sampling [9] uses dataset containing either all datapoints or one random datapoint in a generation. The selection of any case of sample is dependent on probability P%. The author tried different values and found 50% and 75% were best in removing overfilling and improving generalization. Random sampling technique as discussed in Gonçalves et al. [10] used 60% of the data from 3 datasets only and concluded that he obtained fitness close to Standard GP with more generalized solutions. However, both of these results with respect to fitness were contradicted in Žegklitz and Pošík [23] with experimentation on 6 datasets and found that another technique RST-R (random sized sample and points) was the best among these sampling techniques; however it was still not good against SGP w.r.t fitness. The author also divided data further into validation sets for all sampling techniques and observed that validation sets does not help much. Because of the random behavior results varied a lot in each run which led us to believe that there may be some data points which would help more to achieve better results. In Giacobini et al. [8] author proved with experimentation that sample should not have data-points less than entropy w.r.t classes and that sample data should be properly distributed.

Micheal in Kommenda et al. [11] used variable training sets and migrated these sets with other populations after some generations. However, each population had its own fixed training set too. The objective was generalization and reduction in computation cost which he had achieved with Data migration with Multi-Population. But like in Žegklitz and Pošík [23] author observed that with only data migration the results were not better than SGP. Michaela in Sikulova et al. [21] reduced computation cost via fitness predictor sampling that predicts fitness like standard GP.Fitness predictors are dependent on another machine learning algorithm like Genetic Algorithm which is used to co-evolve them for better prediction. This was done using Training object vectors and the variable size array of TOV's. Which were used for evaluation of population being evolved using Cartesian Genetic Programming. This resulted in many times faster evolution. Selection pressure from data points can be modified with either sampling or weighting the data points. Three authors worked on variants to focus data-points on which the population struggles. Mario in Giacobini et al. [8] worked on weighting of data points on which best individual of the generation was struggling to get GP out of a local optima and thus achieve better fitness. The weights were difference between expected and desired output. Another variant here also used constant weight for each point but non-constant weights proved to be better. However in KW-IS Martínez et al. [16] author was inspired by Interleaved sampling and Novelty search ϵ-descriptor from Martínez et al. [17]. In KW-IS each generation uses entire dataset in every other generation while for the rest of generation's algorithm uses top P_{10}% of the ϵ-descriptor. meaning the most difficult points for the population. Lacava in La Cava et al. [12] applied Lexicase selection to regression problems in which best individual from population for any random point is selected as parent in each selection call. However, He observed that this method is not very good for regression problems because the criteria is too stringent. So, he proposed ϵ-Lexicase with 4

definitions of ϵ, two with thresholding hyper parameters while two vaiants with adaptive thresholding based on median. Unlike Yuliana in Spector [22], Lacava compared these with parent selection methods like random selection ,tournament selection and AEF [20]. He observed that his adaptive variants performed the best. While other two are problem dependent.

Jan in Žegklitz and Pošík [23] compared Backwarding [19], Validation Start, RST 1/1, Random Interleaved (RI) [9], RST-R and aforementioned all with combination of validation sets. For a training set with subsets A and B. In backwarding training is driven by subset A while model is selected using subset B. In Validation Start, both evolution and model selection is done using training sets A and B. However, The fitness calculation is weighted such that the weight of both sets is always equal to one but they change continuously with the given condition. RST 1/1 is an extreme sampling case which uses only single random data point in a generation. Author concluded after experimentation that random sized datasets might be more beneficial then single-element or interleaved sampling. Jan also concluded that random sampling can be highly data dependent and may or may not provide better results. While adding validation sets to all those sampling techniques. He concluded that using validation do not significantly improve the results. Yuliana also compared fitness case sampling methods in Martínez et al. [18]. The survey was done on IS, RIS, Lexicase Selection (LEX) and KW-IS with both regression and classification problems. For classification LEX [22] and KW-IS showed similar performance while IS and RIS showed poor performance on classification tasks. On the other hand, for regression problems Standard GP (SGP) outperformed all other sampling techniques for simpler benchmark problems. While for complex real world problems these methods are significantly better than SGP. They also show exhibit substantially smaller amounts of over-fitting and produce smaller program sizes then SGP.

Dhiman and Kumar [3] proposed EPO algorithm. EPO was applied to multiple CSP problems. To solve these problems, multiple types of penalty functions were defined. These include Static penalty, Dynamic penalty, Annealing penalty, Adaptive penalty, Co-evolutionary penalty, and Death penalty. These penalties apply different level of penalties in case of constrain violations. For example, in Death penalty zero fitness was assigned to the violator. In Dhiman and Kumar [2, 4, 5], introduced MOSHO algorithm for solving multi objective problems. It is includes two components. The first component is an archive which is accountable for storing the best non-dominated Pareto optimal solutions. The second component is a group selection approach to choose nearby promising solutions.

Darwaish et al. [1] proposed an approach named DDGP where it was highlighted that due to selection pressure in selecting the right ranges GP stagnates. Due to which GP is unable to find the target solution. An error change parameter was introduced that monitors stagnation. If the best individual does not improve for specific number of generations. Stagnation is declared, evolution process is stopped, all states are stored, target vector is updated with error vector and a sub run is started to evolve the updated target vector. This recursive process avoids stagnation and GP bloat. In DDGP by examining it was observed that there are points where GP readily found the solution. However, on problematic areas the GP stagnates and could not evolve further. As a result only problematic data points have some output values resulting in focus on these points only. And finally all solutions are merged.

Table 1 Benchmark equations used in experiments

#	Equation	Points	Range
1.	$0.3xsin(2\pi x)$	200	$[-1, 0.01, 1]$
2.	$x^3exp^{-x}cos(x)(sin(x)0.35x(cos(x)-1))$	200	$[-1, 0.01, 1]$
3.	$x^5 - 2x^3 + x$	200	$[-1, 0.01, 1]$
4.	$x^6 - 2x^4 + x^2$	200	$[-1, 0.01, 1]$
5.	$sin(x^2)cos(x) - 1$	200	$[-1, 0.01, 1]$

Range denoted by [Start, Step, End]

3 Standard GP struggles on inflexion points

Our experiments have shown that GP fails to evolve inflexion points but readily evolves simple points of the given dataset. Five different non-linear regression problems with varying complexity were studied and are mentioned in Table 1. In Table 1, x is the independent variable sampled at 200 points. The sampling range is defined in the form [*Start*, *Step*, *End*] ("Range" column of Table 1), where *Start* and *End* are the starting and ending values of the sampling, while *Step* is the step size for the sampling. For each problem, training data set is generated by sampling the polynomial from -1 to 1 with step size of 0.01. Standard GP was run 30 times on the training data set using standard setup and output of the best-of-run was recorded. These runs generate 30 data points against each input value. The generated data is skewed therefore box plots are used for visualization. These plots are shown in Fig. 2. Outliers are shown with a circle. Same set of parameters were used for all the GP runs and are mentioned in Table 2.

The box plots show the five-number summary (Min, Q1, Median, Q3, Max) for the distribution generated by GP for each data point. Interquartile range IQR (Q3–Q1) for a point describes the effort put in by GP to evolve this point. In this study we define the struggle put in by GP to find consistently good result on a point by IQR value. Higher IQR value

Table 2 Run parameters for GP

Parameter	Value
No. of runs per experiment	30
No. of generations	50
Population size	2500
Crossover rate	0.9
Mutation rate	0.1
Reproduction rate	0.0
Maximum depth	17
Generations for stagnation	3
Difficulty classes	3
Selection method	Roulette wheel
Functional set	$+, -, *, /, \sin, \cos, \exp, \log$

means bigger effort and vice versa. The rationale behind this is that GP search becomes exploration on difficult to evolve points and at times improve fitness by, mainly fitting easy to evolve points. It is easy to conclude from Fig. 2 that GP has struggled the most on equations 1 and 4, while has evolved Eq. 2 easily. In Fig. 2a, the region of the curve from $-0.62 \geq x \leq 0$ is a difficult region due to high IQR values in this range. Same is evident by the number of the outliers generated within this range. Same explanation is applicable to other plots without exception. Further analysis of the plots suggest that difficult to evolve points have high rate of change of tangents. In this work we use this observation to identify difficult to evolve points. Later GP is forced to evolve these points to improve its performance.

4 Proposed DFS-GP

Our proposed technique is a two-step process. In the first step, preprocessing step, *difficult regions* of the given datasets are marked. Then the points from these regions are introduced by using two new sampling techniques.

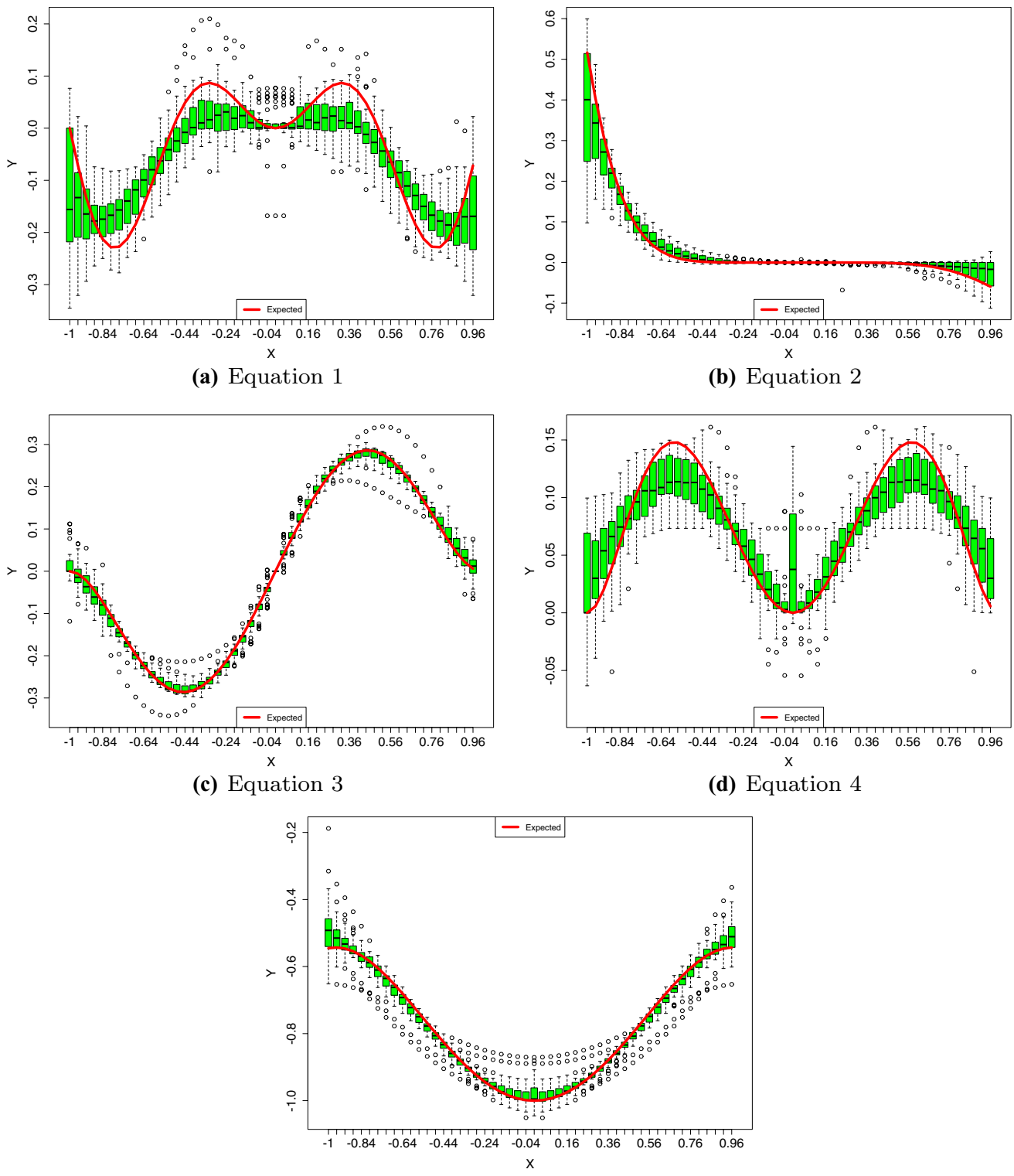
4.1 Identification of difficult regions

In order to identify difficult regions within the given dataset, a concept of *difficulty vector* (DV) is introduced. DV is a set of subsets, where each subset represents the difficulty level. The members of a subset are the index of the data set points which have the difficulty level associated with the subset. Hence $|DV|$ represents the count of difficulty levels. DV_1 represents the easiest set of points, DV_2 relatively difficult and so on. The final DV will look like $DV = \{\{i_1, i_2, i_3, \ldots, i_k\}, \{j_1, j_2, j_3, \ldots, j_l\} \ldots, \{k_1, k_2, k_3, \ldots, k_m\}$. The count of data points in each subset may differ and is dictated by the contour of the to-be-evolved expression. Furthermore, difficulty level subsets are mutually exclusive, i.e. $\bigcap_{i=1}^{diff_levels} DV_i = \emptyset$.

Complete algorithm for identifying difficult regions is mentioned as Algorithm 1. As a first step, the data points are arranged in ascending order on the basis of the independent variable. In the next step, slope vector (SV) comprising of slope of each data point is calculated. Then the rate of change of slope vector (SV') is calculated to capture the change in the curve contour. Then SV' is normalized in the range [0,1] and absolute values are used as we are only interested in the magnitude of change. Then difficult regions (Q) are calculated from SV'. Count of the difficult regions ($|Q|$) is a hyper parameter and is set before the run. The points in Q_1 are marked as the easiest points, while the points in $Q_{|Q|}$ are marked as the hardest points to evolve.

Algorithm 1 Algorithm for identifying difficult region

```
function DifficultyVector(Dataset D,count)
    Sort(D)                                   ▷ Sort according to independent var
    for i = 2 : |D| do
        SV_(i) = (y_i − y_{i−1})/(x_i − x_{i−1})
    for i = 2 : length(SV) do
        SV'_(i) = (SV_(i) − SV_(i−1))/(x_i − x_{i−1})
    Q = Calculate_Regions(SV', count)
    DV = ∅
    for i = 2 : |Q| do
        DV_i = ∀x ∈ SV', Q_{i−1} ≤ x ≥ Q_i | index(x)
        DV = DV ∪ DV_i
    return DV
```



(a) Equation 1

(b) Equation 2

(c) Equation 3

(d) Equation 4

(e) Equation 5

Fig. 2 Boxplots of best-of-run of 30 runs. Difficult to evolve points are shown with high IQR values

Lines 3–4 calculate the slope at each data point of the original data set. Lines 5–6 calculate the change in slope with respect to independent variable. Lines 9–11 create final DV by adding individual difficulty level subsets.

4.1.1 Complexity analysis

The algorithm takes $\Theta(|D|log(|D|)$ in the worst case as it involves sorting the values. Normally, data sets are not bigger than a few hundred points hence the process of identification of the difficulty region is very efficient in practice. This process is done as a preprocessing step and is not repeated again. Line by line complexity analysis of the proposed algorithm is as follows.

Line 1 In Worst case takes $\Theta(|D|log(|D|)$

Line 3–4 Slope calculation takes constant time, therefore overall time complexity is linear, i.e. $\Theta(|D|)$

Line 5–6 Takes linear time i.e. $\Theta(|D|)$ as $length(SV) = |D|$

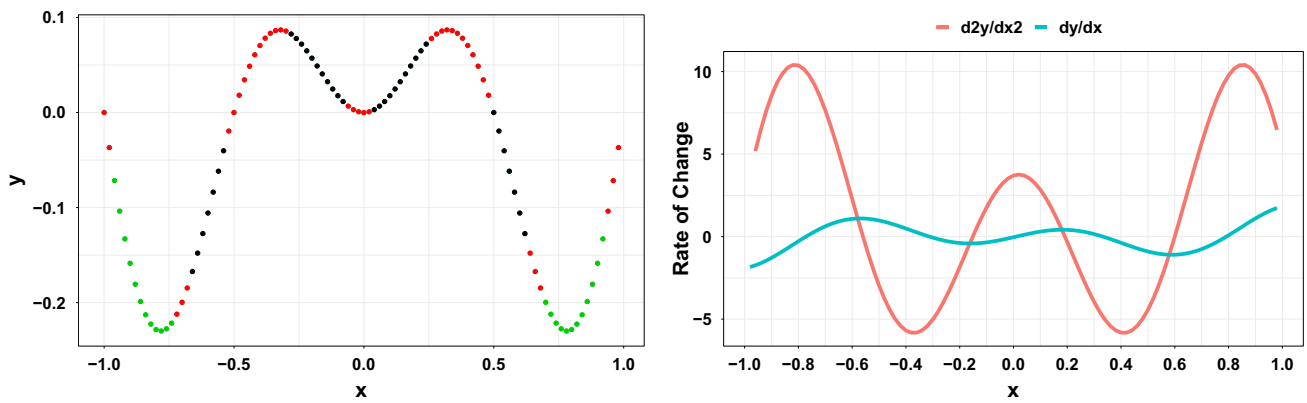
Line 7 Takes constant time to calculate the boundaries of the regions. $\Theta(1)$

Line 9–11 Takes linear time to distribute $|D|$ points in difficulty regions. Note, $|Q| = |D|$

Overall complexity is given by $\Theta(|D|log(|D|) + \Theta(|D|) + \Theta(1) + |D| = \Theta(|D|log(|D|)$

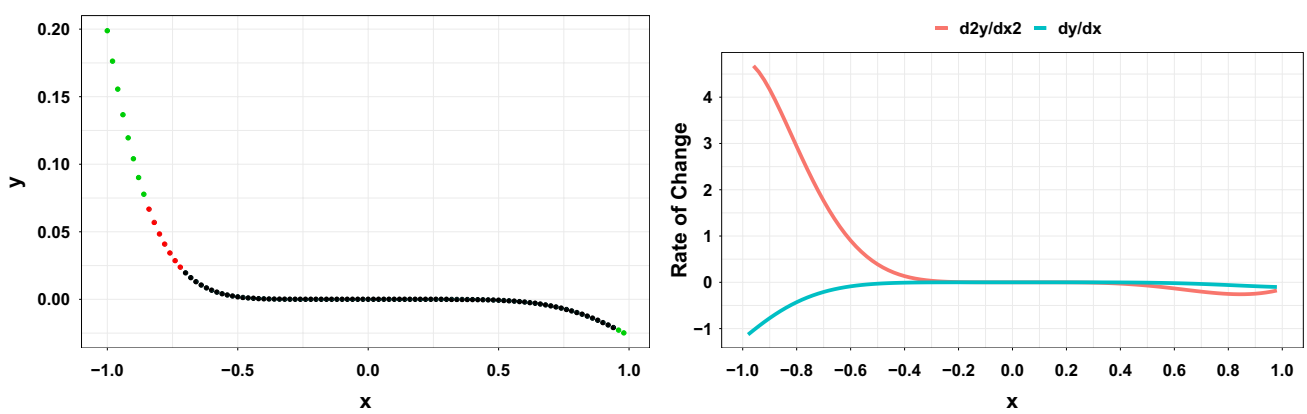
4.1.2 Example

Let's assume that we want to calculate three difficulty levels ($|Q| = 3$) for the data set with 10 points. Let's say



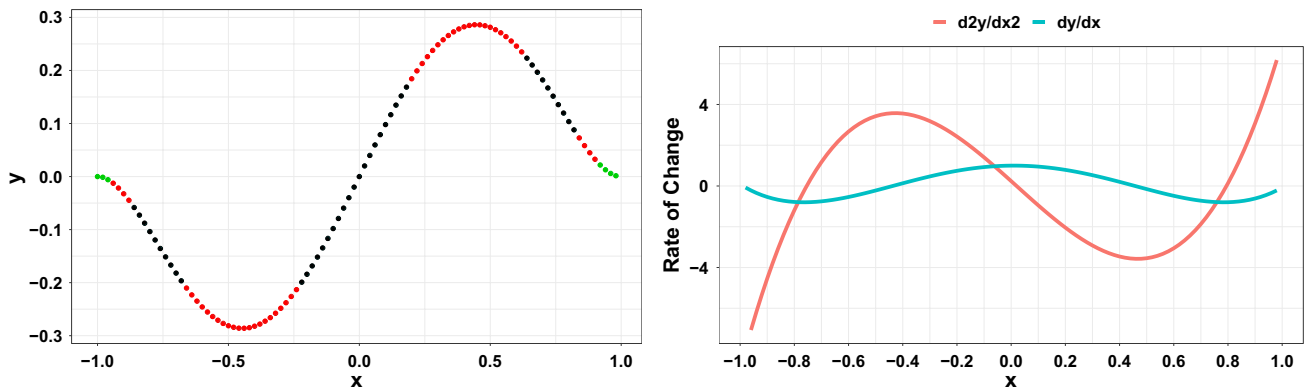
(a) Difficulty Plot **(b)** Gradient Plot

Fig. 3 Plots for Eq. 1. In **a** easiest points are plotted with black color, intermediate level difficult points with red color and the most difficult points with green color. **b** shows the slope (dy/dx) and rate of change of slope (d^2y/dx^2) at each point (color figure online)



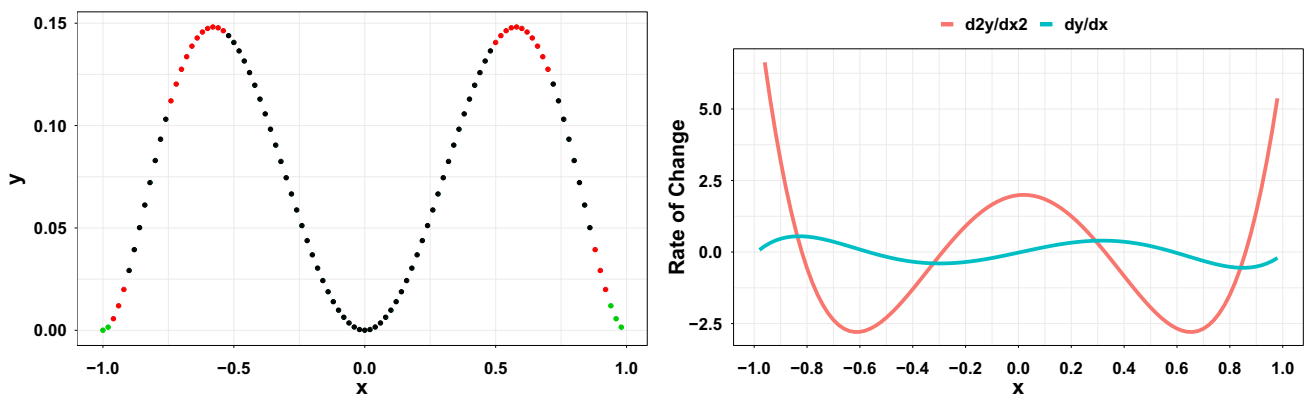
(a) Difficulty Plot **(b)** Gradient Plot

Fig. 4 Plots for Eq. 2. In **a** easiest points are plotted with black color, intermediate level difficult points with red color and the most difficult points with green color. **b** shows the slope (dy/dx) and rate of change of slope (d^2y/dx^2) at each point (color figure online)



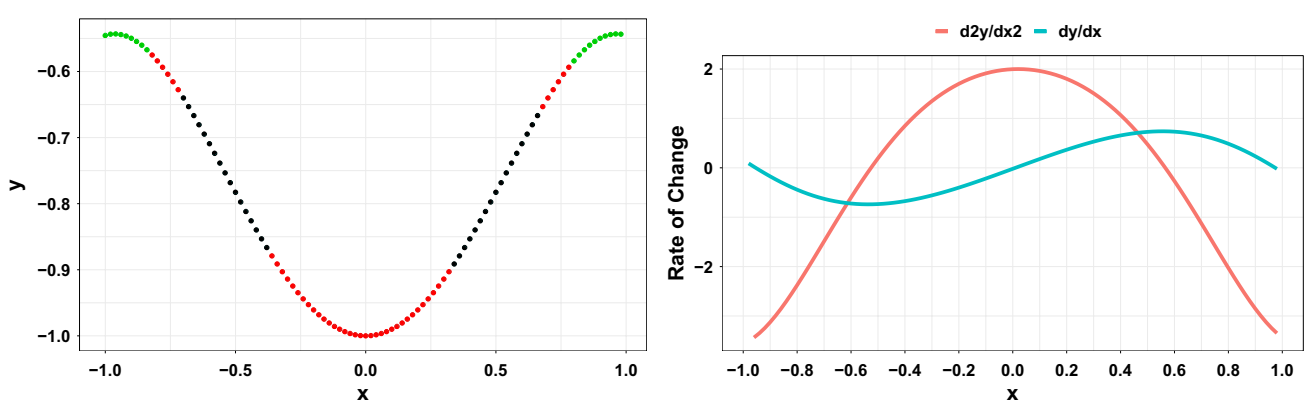
(a) Difficulty Plot **(b)** Gradient Plot

Fig. 5 Plots for Eq. 3. In **a** easiest points are plotted with black color, intermediate level difficult points with red color and the most difficult points with green color. **b** shows the slope (dy/dx) and rate of change of slope (d^2y/dx^2) at each point (color figure online)



(a) Difficulty Plot **(b)** Gradient Plot

Fig. 6 Plots for Eq. 4. In **a** easiest points are plotted with black color, intermediate level difficult points with red color and the most difficult points with green color. **b** shows the slope (dy/dx) and rate of change of slope (d^2y/dx^2) at each point (color figure online)



(a) Difficulty Plot **(b)** Gradient Plot

Fig. 7 Plots for Eq. 5. In **a** Easiest points are plotted with black color, intermediate level difficult points with red color and the most difficult points with green color. **b** shows the slope (dy/dx) and rate of change of slope (d^2y/dx^2) at each point (color figure online)

$SV = \{-0.19, -0.63, -0.72, -0.54, -0.2, 0.2, 0.54, \}$ $SV = \{0.72, 0.63, 0.19\}$. Then $SV' = \{-2.17, -0.47, 0.91, 1.71, 1.96, 1.71, 0.91, -0.47, -2.17\}$. SV' after normalization will be $\{1, 0.22, 0.42, 0.79, 0.9, 0.79, 0.42, 0.22, 1\}$. Ranges for the quantiles are calculated and they turn out to be $\{0.22, 0.42, 0.83, 1\}$. Q_1 has the indices of the points in SV' having values between 0.22 and 0.42 which are 2nd, 3rd , 7th and 8th points of the data set. Similarly, Q_2 has the indices of the points in SV' having values between 0.42 and 0.83, which turns out to be 4th and 6th points of the data set. Rest of the points belong to Q_3. Final DV is {{ 2nd, 3rd, 7th, 8th }, {4th, 6th }, {1st, 5th, 9th, 10th}}.

Difficult regions of the expressions, mentioned in Table 1 are calculated using Algorithm 1 and are graphically shown as Figs. 3, 4, 5, 6 and 7. For this study the count of the difficult regions is set to three. In the figures, easiest points are plotted with Black color, intermediate level difficult points with Red color and the most difficult points with Green color. Gradient plots for the expressions are also mentioned along side. These plots include the slope of the equation (dy/dx) and the rate of change of slope with respect to independent variable (d^2y/dx^2). Regions with d^2y/dx^2 equals to or close to zero means that there is no change in the contour while, higher values of d^2y/dx^2 mean bigger change in the contour. d^2y/dx^2 assigns different difficulty levels to the data points. Low d^2y/dx^2 is mapped to low difficulty and vice versa.

4.2 Sampling techniques

In the next step two different sampling techniques are introduced to select points from the Difficulty Vector (DV) . First one is "Difficult Points First" while the second one is "Difficulty Proportionate Selection". Detail of each is as follows.

4.2.1 Diificult points first (DPF)

In this sampling technique, the hardest points are added for training as the first step. This is to force GP to evolve the most difficult regions of the curve first. After adding the points, GP is allowed to run until stagnation in fitness improvement is observed. At this point, the next lower difficulty level data points are introduced. This way the evolution process continues until complete data is introduced and termination criteria is met. In this technique, fixed number of generations are used for identifying stagnation. Number of generations is a hyper parameter and is set before the run. DPF is a greedy technique with bias towards picking the most difficult points.

4.2.2 Difficulty propotionate selection (DPS)

This technique like DPF is also biased towards the difficult regions but the probability of selection of less difficult regions is not zero in the initial stages unlike DPF.

The DPS method is inspired by RST-R. In our extended approach each point is selected with a probability [0-*difficulty_levels*]. DPS is formally defined in (1).

$$\forall d \in D, \begin{cases} Sampled & diff(d) > rand(diff_levels) \\ Not\ Sampled & Otherwise \end{cases} \tag{1}$$

where d is a data point of the given dataset D and *diff_levels* is the maximum difficulty value set while calculating the difficulty vector DV. The function *diff*(d) reruns the difficulty level of the point d by consulting DV.

4.3 Custom fitness function

In this work, evolutionary runs use only portion of the data set in the initial generations by using above mentioned sampling techniques. It is illogical to apply standard fitness functions, which calculate fitness on complete data set, to calculate the performance of individuals during these runs. In order to tackle this issue, two variants of the standard fitness function are proposed in which fitness of an individual is calculated on the seen data points only. Note that during the run of the proposed techniques, the fitness of a population can only be compared with standard GP run after introduction of the complete data set. Therefore, for fair comparison results are calculated at the end of runs.

4.3.1 Weighted sampled fitness function 1 (W1)

Commonly known standard adjusted fitness is calculated by (2)

$$STD = \frac{1}{1 + \sum_{i=1}^{n} (Y_{exp_i} - Y_{act_i})^2} \tag{2}$$

where n is no. of fitness cases and $(Y_{exp_i} - Y_{act_i})$ is the error at data point i. We have introduced an improved version of STD which is mentioned in (3).

$$W1 = \frac{1 - w}{1 + \sum_{j=1}^{m} error_j^2} + \frac{w}{m - hits} \tag{3}$$

where, m is the number of data points introduced at some generation. Note, that $0 \leq m \leq n$ and $m \rightarrow n$ with time. w is weight parameter with value between 0 and 1. In this work, w is inversely proportional to amount of data introduced. This means w diminishes with the introduction of new data points. Note, when $m = n$, $w \approx 0$ (3) behaves like (2). At this point performance of the proposed technique is comparable

with Standard GP. The part after summation in (3) ensures that good individuals found in the early part of the run are preserved throughout the run.

4.3.2 Weighted sampled fitness function 2 (W2)

Main limitation of $W1$ is that it introduces new data on regular intervals by assuming that GP improves its fitness on regular basis which is not true in reality. Keeping this in mind an improved weighted fitness function $W2$ is introduced. $W2$ is given in (4).

$$W2 = \frac{v}{1 + \sum_{j=1}^{m} error_j^2} + \frac{1 - v}{1 + \sum_{j=1}^{m} diff(j) error_j^2} \tag{4}$$

where $v = current_generation / total_generation$. $diff(j)$ returns the difficulty level of data point j. Like before, as time passes, $v \rightarrow 1$ and $m \rightarrow n$. This reduces (4) to (2) over time. Second part of Eq. (4) puts parsimony pressure on the system to evolve difficult points first.

5 Experimental setup

Four independent experiments were run to test our hypothesis. In the experiments, performance of GP augmented with different sampling techniques and fitness functions was compared with KW-IS, RSTR and Standard GP. Number of generations was set to 3 for identifying stagnation in fitness. Student t-test was run to check the significance of the performance difference. Detail of each setup is as follows:

DPF-W1 DPF along with $W1$ fitness function was used during the run.

DPF-W2 DPF along with $W2$ fitness function was used during the run.

All-W2 No sampling technique was used and complete data set along with $W2$ fitness function was used during the run. Furthermore, m was set to n for this experiment.

DPS-STD In this experiment DPS (see Sect. 4.2.2 for details) was used for sampling and standard adjusted fitness function [mentioned as Eq. (2)] was used. For this setup, n in (2) was replaced with m. Meaning of m is the same, i.e. number of data points introduced in a generation.

The five symbolic regression problems mentioned in Table 1 were solved in these experiments. Hyper parameter settings of Table 2 were used for all the experiments for fair comparison.

6 Results and discussions

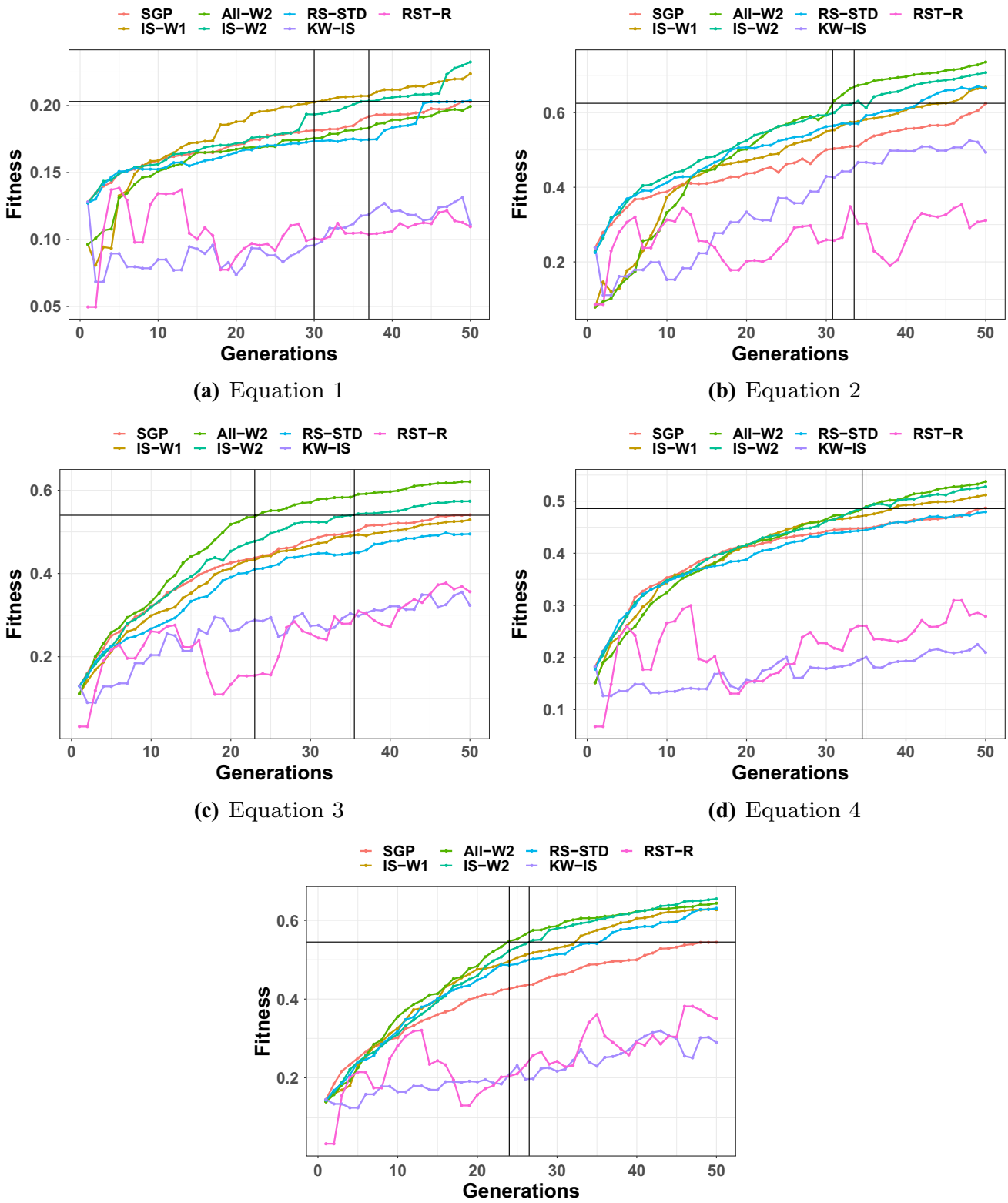
Result of the experiments are shown in Table 3. The table entries are the mean accuracy of 30 best-of-run individuals for each setup. As mentioned earlier, the basic parameters for all the experiments were kept same for fair comparison.

From Table 3 it is clear that the proposed setups are better than other setups. Standard GP is the second best performer after the proposed setups. The results of the significance test between the proposed setups and Standard GP are included in Table 3 and are shown within parentheses. "Yes" means that the performance difference between the proposed setup and GP is significant, while, "No" means that the difference is insignificant. The best performers are shown bold.

Results show that the setups using fitness function $W2$ are performing the best. On certain occasions our results are not significantly better than Standard GP but in none of the occasions they are inferior than Standard GP. *ALL-W2* is significantly better on equations 2 and 3. Whereas, *DPF-W2* is showing slightly better results for the rest of the equations.

Table 3 Results for mean absolute fitness of all the setups. Results of t-test are shown within parantheses

	SGP	KW-DPF	RSTR	DPF-W1	DPF-W2	All-W2	DPS-STD
Eq-1	0.203473	0.114312	0.109567	0.225385 (No)	**0.233285 (No)**	0.200005 (No)	0.205615 (No)
Eq-2	0.617124	0.492448	0.310963	0.666869 (No)	0.704318 (Yes)	**0.728861 (Yes)**	0.653111 (No)
Eq-3	0.545953	0.325005	0.356085	0.531188 (No)	0.575742 (Yes)	**0.623703 (Yes)**	0.493774 (No)
Eq-4	0.485653	0.204597	0.279034	0.509758 (No)	**0.538306 (No)**	0.534566 (No)	0.478152 (No)
Eq-5	0.542641	0.288987	0.349720	0.631666 (No)	**0.65109 (Yes)**	0.643322 (Yes)	0.629332 (No)



(a) Equation 1

(b) Equation 2

(c) Equation 3

(d) Equation 4

(e) Equation 5

Fig. 8 Convergence plots for different setups. Horizonatl lines show the maximum fitness reported by Standard GP. Verticle lines show the evaluations taken by the top two proposed setups to match the best performance of Standard GP

6.1 Proposed technique converges fast

To get an insight into the results, the *convergence speed* of the proposed technique and Standard GP is compared. Convergence speed is defined as the number of generations required to attain given fitness value. Clearly, a setup taking less number of generations to attain given fitness is better (computationally cheap). Convergence plots of all the setups for the five equations are shown in Fig. 8. Note that the proposed setups were at loss in the early generations when only fraction of the data set was introduced for training. In Fig. 8, the annotated horizontal lines show the maximum fitness attained by Standard GP. Two vertical annotated lines are drawn at the intersection of this line with the fitness plots of the top two proposed schemes. These intersection points indicate the number of generations used by top two performing proposed setups to match the best performance of Standard GP. The results show that in the worst case scenario the proposed schemes are 30% faster than Standard GP (For example Eq. 4). On other occasions these schemes are

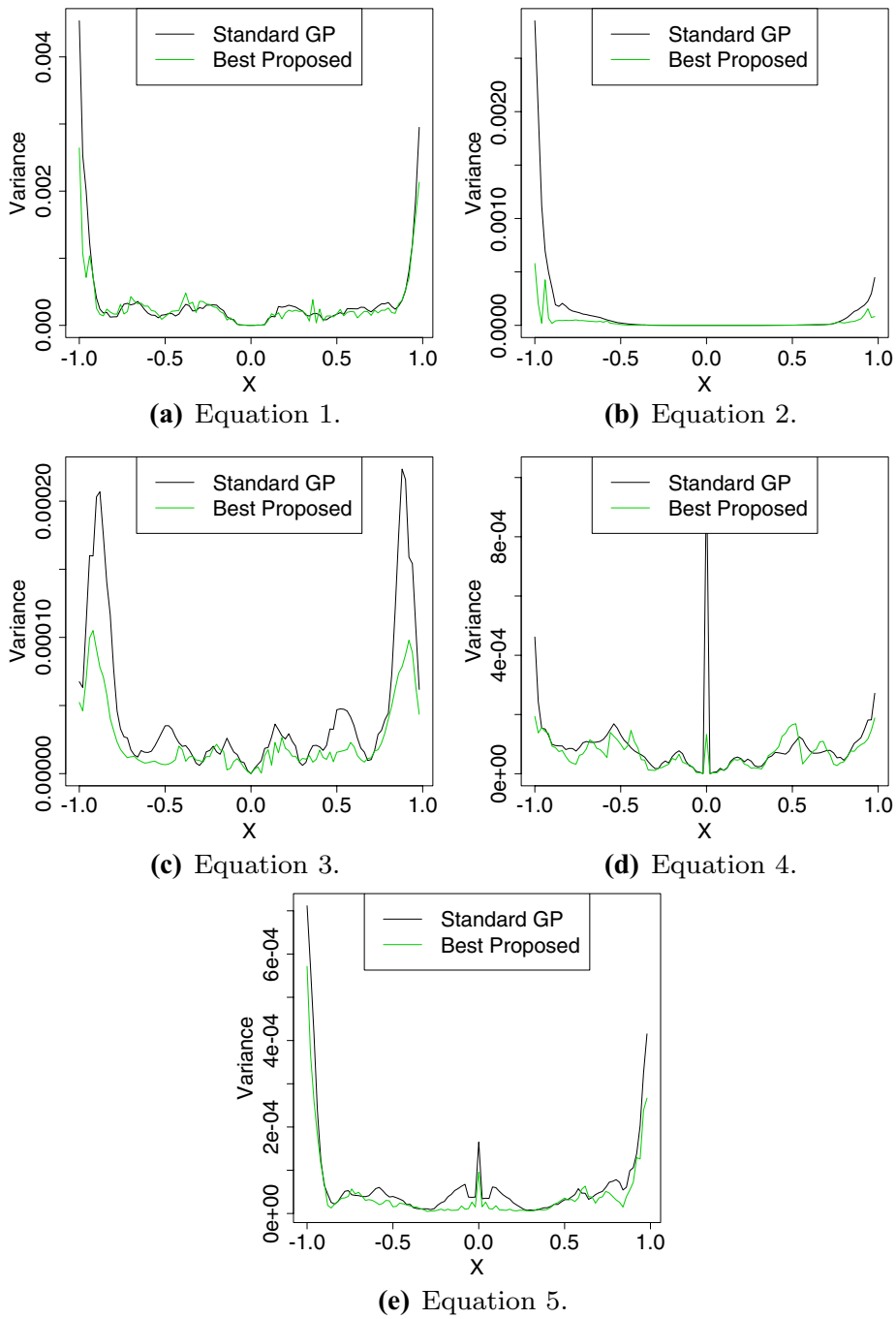


Fig. 9 Comparison of variance plots of the proposed setups with Standard GP

50% faster than Standard GP (For example equations 3 and 5). The performance of the proposed setups are either at par or better than Standard GP throughout the runs. Recall that in the early generations our setups had seen only fraction of the training set. This observation suggests that our technique has the ability to generalise better than Standard GP.

6.2 Proposed technique has evolved superior solutions

Finally, to compare the quality of the results, the variance in output distribution of the best-of-run individuals against each data point was recorded for all the setups. These values are plotted and shown in Fig. 9. For the sake of brevity only the best performing setup is shown. Outliers are excluded from the plots to include the effect of the majority elements. Low variance means that the evolved individuals are close/similar to each other and vice versa. Spikes in the plots show the variance in output values against hard data points. In all the cases, the proposed setups are showing small variances and fit the expected curves better than Standard GP. This difference is quite significant on the hard points. For example in Fig. 9b the proposed setup is performing better on the left and right margins. Similar behaviour is shown by all the proposed setups. For some equations the variance difference between the proposed setup and Standard GP setup is large (look at Fig. 9c). This is consistent across all the equations.

7 Conclusion

In this paper a simple and effective technique was proposed to improve the performance of Standard GP. It is shown that the given input dataset, generally has easy and difficult points. Normally, difficult points are in minority and are mostly ignored by GP while improving its performance. Furthermore, we showed that the behaviour of GP can be improved by augmenting GP with two stage technique. Gradient of the curve was used to define difficult to evolve points on the curve. These points were introduced during the run at different stages of a run to ensure that GP evolved each point correctly before moving on to next point. Effectiveness of the proposed technique was demonstrated on three different departments, namely, accuracy, convergence speed and goodness of fit. The proposed technique has shown superior performance on all the departments. Moreover, the convergence speed result has shown the possibility of evolving superior results by training only on the fraction of the training set.

Currently, we are working on the extension of the current work so that it can be applied to multivariate expressions. Further experimentation is required to tweak hyper parameters like, number of generations to identify stagnation and the number of generations. We reckon that number of generations can be reduced by looking at the stagnation in a run.

References

1. Darwaish A, Majeed H, Ali MQ, Rafay A (2017) Dynamic programming inspired genetic programming to solve regression problems. Int J Adv Comput Sci Appl. https://doi.org/10.14569/IJACSA.2017.080463
2. Dhiman G, Kaur A (2017) Spotted hyena optimizer for solving engineering design problems. In: 2017 international conference on machine learning and data science (MLDS), pp 114–119. https://doi.org/10.1109/MLDS.2017.5
3. Dhiman G, Kumar V (2018a) Emperor penguin optimizer: a bio-inspired algorithm for engineering problems. Knowl Based Syst 159:20–50. https://doi.org/10.1016/j.knosys.2018.06.001
4. Dhiman G, Kumar V (2018b) Multi-objective spotted hyena optimizer: a multi-objective optimization algorithm for engineering problems. Knowl Based Syst 150:175–197. https://doi.org/10.1016/j.knosys.2018.03.011
5. Dhiman G, Kumar V (2019) Spotted hyena optimizer for solving complex and non-linear constrained engineering problems. In: Yadav N, Yadav A, Bansal JC, Deep K, Kim JH (eds) Harmony search and nature inspired optimization algorithms. Springer, Singapore, pp 857–867
6. Doucette J, Heywood MI (2008) GP classification under imbalanced data sets: active sub-sampling and AUC approximation. In: O'Neill M, Vanneschi L, Gustafson S, Esparcia Alcázar AI, De Falco I, Della Cioppa A, Tarantino E (eds) Genetic programming. Springer, Berlin, pp 266–277
7. Gathercole C, Ross P (1994) Dynamic training subset selection for supervised learning in genetic programming. In: Davidor Y, Schwefel HP, Männer R (eds) Parallel problem solving from nature—PPSN III. Springer, Berlin, pp 312–321
8. Giacobini M, Tomassini M, Vanneschi L (2002) Limiting the number of fitness cases in genetic programming using statistics. In: Guervós JJM, Adamidis P, Beyer HG, Schwefel HP, Fernández-Villacañas JL (eds) Parallel problem solving from nature—PPSN VII. Springer, Berlin, pp 371–380
9. Gonçalves I, Silva S (2013) Balancing learning and overfitting in genetic programming with interleaved sampling of training data. In: Krawiec K, Moraglio A, Hu T, Etaner-Uyar AŞ, Hu B (eds) Genetic programming. Springer, Berlin, pp 73–84
10. Gonçalves I, Silva S, Melo JB, Carreiras JM (2012) Random sampling technique for overfitting control in genetic programming. In: European conference on genetic programming. Springer, pp 218–229
11. Kommenda M, Affenzeller M, Burlacu B, Kronberger G, Winkler SM (2014) Genetic programming with data migration for symbolic regression. In: Proceedings of the companion publication of the 2014 annual conference on genetic and evolutionary computation. ACM, pp 1361–1366
12. La Cava W, Spector L, Danai K (2016) Epsilon-lexicase selection for regression. In: Proceedings of the genetic and evolutionary computation conference 2016, ACM, New York, NY, USA, GECCO '16, pp 741–748. https://doi.org/10.1145/2908812.2908898,
13. Lasarczyk CW, Dittrich P, Banzhaf W (2004a) Dynamic subset selection based on a fitness case topology. Evolut Comput 12(2):223–242. https://doi.org/10.1162/106365604773955157
14. Lasarczyk CWG, Dittrich P, Banzhaf W (2004b) Dynamic subset selection based on a fitness case topology. Evolut Comput 12(2):223–242. https://doi.org/10.1162/106365604773955157

15. Majeed H, Ryan C (2007) On the constructiveness of context-aware crossover. In: Proceedings of the 9th annual conference on genetic and evolutionary computation, ACM, New York, NY, USA, GECCO '07, pp 1659–1666. https://doi.org/10.1145/1276958.1277286,
16. Martínez Y, Trujillo L, Naredo E, Legrand P (2014) A comparison offitness-case sampling methods for symbolic regression with genetic programming V. In: Tantar AA, Tantar E, Sun JQ, Zhang W, Ding Q, Schütze O, Emmerich M, Legrand P, Del Moral P, Coello Coello CA (eds) EVOLVE—a bridge between probability, set oriented numerics, and evolutionary computation. Springer, Cham, pp 201–212
17. Martínez Y, Naredo E, Trujillo L, Galván-López E (2013) Searching for novel regression functions. In: 2013 IEEE congress on evolutionary computation, pp 16–23. https://doi.org/10.1109/CEC.2013.6557548
18. Martínez Y, Naredo E, Trujillo L, Legrand P, López U (2017) A comparison of fitness-case sampling methods for genetic programming. J Exp Theor Artif Intell 29(6):1203–1224. https://doi.org/10.1080/0952813X.2017.1328461
19. Robilliard D, Fonlupt C (2002) Backwarding: an overfitting control for geneticprogramming in a remote sensing application. In: Collet P, Fonlupt C, Hao JK, Lutton E, Schoenauer M (eds) Artificial evolution. Springer, Berlin, pp 245–254
20. Schmidt M, Lipson H (2011) Age-fitness Pareto optimization. Springer, New York, pp 129–146. https://doi.org/10.1007/978-1-4419-7747-2_8
21. Sikulova M, Hulva J, Sekanina L (2015) Indirectly encoded fitness predictors coevolved with cartesian programs. In: Machado P, Heywood MI, McDermott J, Castelli M, García-Sánchez P, Burelli P, Risi S, Sim K (eds) Genetic programming. Springer, Cham, pp 113–125
22. Spector L (2012) Assessment of problem modality by differential performance of lexicase selection in genetic programming: A preliminary report. In: Proceedings of the 14th annual conference companion on genetic and evolutionary computation. ACM, New York, NY, USA, GECCO '12, pp 401–408, https://doi.org/10.1145/2330784.2330846,
23. Žegklitz J, Pošík P (2015) Model selection and overfitting in genetic programming: empirical study. In: Proceedings of the companion publication of the 2015 annual conference on genetic and evolutionary computation. ACM, New York, NY, USA, GECCO companion '15, pp 1527–1528. https://doi.org/10.1145/2739482.2764678,

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.